Universidad de Granada

ETSIIT Informática y Telecomunicación

Dpto. de Ciencias de la Computación e Inteligencia Artificial

Metaheurísticas

Práctica 1

Técnicas de Búsqueda Local y Algoritmos Greedy
para el Problema de Portfolio

Curso 2025–2026

Nombre: Jose Angel Carretero Montes

DNI: 25628892S

Email: joseangel1@correo.ugr.es

Grupo: Grupo 3

Índice

1. Descripción del Problema **2**

2. Aplicación de los Algoritmos al Problema **2**
2.1. Representación de soluciones 2
2.2. Generación aleatoria de soluciones 3
2.3. Operador de reparación (`fix`) 3
2.4. Precómputo de matrices 3

3. Algoritmos **3**
3.1. Algoritmo Aleatorio (*Random Search*) 3
3.2. Algoritmo Greedy 4
3.3. Búsqueda Local (BL) 4
3.4. Simulated Annealing (SA) — Trabajo voluntario 5

4. Estructura del Código y Manual de Usuario **6**
4.1. Estructura de directorios 6
4.2. Compilación 7
4.3. Ejecución 7

5. Experimentos y Resultados **7**
5.1. Casos del problema 7
5.2. Resultados 8
5.3. Boxplots 8

6. Análisis de Resultados **10**
6.1. Calidad de las soluciones 10
6.2. Variabilidad 10
6.3. Coste computacional 10

1. Descripción del Problema

El **Problema de Portfolio** consiste en determinar la asignación óptima de capital entre N activos financieros con el objetivo de maximizar el retorno esperado minimizando simultáneamente el riesgo.

Representación de los datos

Para cada activo $i \in \{1, \ldots, N\}$ y día $t \in \{1, \ldots, T\}$ se dispone del retorno diario μ_{ti}, calculado como el ratio de mejora relativa del valor:

$$\mu_{ti} = \frac{V_{ti} - V_{(t+1)i}}{V_{ti}}$$

Los datos empleados cubren el periodo 2015–2024 para entrenamiento y el año 2025 para evaluación de generalización.

Función objetivo

Dado un vector de pesos $w = (w_1, \ldots, w_N)$, la calidad de la cartera se mide como:

$$\text{fitness}(w) = \text{Beneficio}(w) - \lambda \cdot \text{Riesgo}(w)$$

donde:

$$\text{Beneficio}(w) = \sum_{i=1}^{N} w_i \cdot \frac{1}{T} \sum_{t=1}^{T} \log(1 + \mu_{ti}) \qquad \text{Riesgo}(w) = \sum_{i=1}^{N} \sum_{j=1}^{N} w_i \cdot \Sigma_{ij} \cdot w_j$$

siendo $\Sigma_{ij} = \frac{1}{T} \sum_{t=1}^{T} (\mu_{ti} - \bar{\mu}_i)(\mu_{tj} - \bar{\mu}_j)$ la covarianza entre activos i y j, y $\lambda = 500$ el factor de penalización del riesgo.

Restricciones

$$\sum_{i=1}^{N} w_i = 1 \qquad w_i = 0 \ \vee \ lo \leq w_i \leq hi \quad \forall i \in 1 \ldots N$$

Los rangos $[lo, hi]$ varían según el conjunto de datos (véase Sección 5).

2. Aplicación de los Algoritmos al Problema

2.1. Representación de soluciones

Una solución queda representada como un vector $w \in \mathbb{R}^N$ donde cada componente w_i indica la fracción del capital invertida en el activo i. La restricción de cardinalidad implica que w_i puede ser exactamente cero (no se invierte en esa empresa) o estar en el intervalo $[lo, hi]$.

2.2. Generación aleatoria de soluciones

Para construir una solución aleatoria válida se procede como sigue:

1. Se elige aleatoriamente un número de empresas en las que *no* se invierte, $n_0 \in [1, N-1]$.
2. Los índices restantes $n_+ = N - n_0$ se seleccionan barajando un vector de índices.
3. A cada empresa activa se le asigna un peso aleatorio en $[lo, hi]$.
4. Se normaliza: $w_i \leftarrow w_i / \sum_k w_k$.
5. Se llama al operador `fix()` para garantizar factibilidad.

2.3. Operador de reparación (`fix`)

Tras cualquier modificación de la solución puede violarse alguna restricción. El operador de reparación actúa en tres fases:

1. **Normalización:** $w_i \leftarrow w_i / \sum_k w_k$.
2. **Recorte de rango:** si $0 < w_i < lo$ entonces $w_i \leftarrow 0$; si $w_i > hi$ entonces $w_i \leftarrow hi$.
3. **Redistribución iterativa:** mientras $|\sum w_i - 1| > \varepsilon$:
   - Si falta dinero por invertir: se reparte la diferencia entre las empresas activas con $w_i < hi$. Si no hay candidatos, se activa una empresa inactiva asignándole lo.
   - Si sobra dinero: se retira la diferencia de las empresas con $w_i > lo$.

2.4. Precómputo de matrices

Para acelerar la evaluación, se precalculan antes de ejecutar los algoritmos: el vector de factores de beneficio $b_i = \frac{1}{T}\sum_t \log(1 + \mu_{ti})$ y la matriz de covarianza Σ, tanto para los datos de entrenamiento (2015–2024) como para los de test (2025). Así, `fitness` se calcula en $O(N^2)$.

3. Algoritmos

3.1. Algoritmo Aleatorio (*Random Search*)

Genera `MAX_EVALS` soluciones aleatorias independientes y devuelve la mejor. Sirve como referencia inferior de calidad.

Algorithm 1: Búsqueda Aleatoria

Entrada: problema, maxEvals
Salida: mejor solución encontrada

```
1 mejorFit ← −∞;
2 para i ← 1 to maxEvals hacer
3 |   s ← crearSolución();
4 |   f ← fitness(s);
5 |   si f > mejorFit entonces
6 |   |   mejorFit ← f;
7 |   |   mejorSol ← s;
8 devolver mejorSol
```

3.2. Algoritmo Greedy

Construye una solución de forma determinista asignando capital en orden decreciente de una función heurística que estima el valor marginal de cada activo.

Función heurística

La heurística para el activo i estima su contribución neta al fitness si fuese el único activo de la cartera, penalizando su covarianza media con el resto:

$$h(i) = b_i - \lambda \cdot \frac{1}{N} \sum_{j=1}^{N} \Sigma_{ij}$$

donde b_i es el factor de beneficio precalculado y Σ_{ij} la covarianza.

Algorithm 2: Greedy

Entrada: problema
Salida: solución construida

```
1 Calcular h(i) para todo i ∈ {1, ..., N};
2 Ordenar empresas en orden decreciente por h(i) → lista companies;
3 w ← 0;   suma ← 0;
4 mientras suma < 1 − ε y quedan empresas en companies hacer
5 |   c ← primera empresa de companies;
6 |   Eliminar c de companies;
7 |   w[c] ← mín(hi, 1 − suma);
8 |   suma ← suma + w[c];
9 devolver w
```

El greedy es determinista: produce siempre la misma solución independientemente de la semilla, por lo que su resultado es idéntico en las 50 ejecuciones.

3.3. Búsqueda Local (BL)

Implementa una búsqueda de **primero mejor** (*first improvement*) sobre el vecindario definido por todos los pares de empresas (p_1, p_2), $p_1 \neq p_2$, con $p_1, p_2 \in \{1, \ldots, N\}$.

Operador de generación de vecino

Dado un par $(origen, destino)$, el vecino se obtiene transfiriendo un $ratio = 0{,}4$ del peso del activo origen al activo destino:

$$\text{change}(w, origen, destino): \quad w'_{origen} = w_{origen} \cdot (1 - ratio), \quad w'_{destino} = w_{destino} + ratio \cdot w_{origen}$$

Tras el cambio se aplica `fix()` para restaurar la factibilidad.

Esquema de la BL

Algorithm 3: Búsqueda Local (primero mejor)

Entrada: problema, maxEvals
Salida: mejor solución encontrada

```
P ← todos los pares (i, j) con i ≠ j, i, j ∈ {1, ..., N};
s ← crearSolución(); f ← fitness(s); evals ← 1;
mejora ← verdadero;
mientras mejora y evals < maxEvals hacer
    mejora ← falso;
    barajar(P);
    para cada (origen, destino) ∈ P hacer
        si evals ≥ maxEvals entonces
            romper;
        s' ← s; s'[origen] ← s'[origen] · 0,6;
          s'[destino] ← s'[destino] + s[origen] · 0,4;
        fix(s'); f' ← fitness(s'); evals += 1;
        si f' > f entonces
            s ← s'; f ← f'; mejora ← verdadero;
            romper;
devolver s
```

El vecindario tiene $N \cdot (N-1)$ vecinos. La exploración se baraja en cada iteración para evitar sesgos de orden. La condición de parada es la convergencia (ninguna mejora en un recorrido completo) o el agotamiento del presupuesto.

3.4. Simulated Annealing (SA) — Trabajo voluntario

El Recocido Simulado extiende la búsqueda local aceptando soluciones peores con probabilidad $e^{\Delta f/T}$, lo que permite escapar de óptimos locales. La temperatura T decrece geométricamente ($T \leftarrow T \cdot \alpha$), lo que crea una transición natural de exploración (alta T) a explotación (baja T).

Calibración automática de temperatura

En lugar de fijar T_0 manualmente, se estima automáticamente realizando 200 movimientos aleatorios, calculando el empeoramiento medio $\overline{\Delta}$ de los movimientos que empeoran, y

fijando:

$$T_0 = \frac{-\overline{\Delta}}{\ln(0{,}8)}$$

de modo que al inicio se acepta aproximadamente el 80 % de los movimientos que empeoran, garantizando exploración amplia independientemente de la escala del problema.

Operador de vecino

Se elige aleatoriamente una empresa *origen* con peso positivo y una empresa *destino* cualquiera distinta. Se transfiere una fracción aleatoria $\delta \in [0{,}1, 0{,}9]$ del peso del origen al destino, seguida de `fix()`. La aleatoriedad de δ proporciona mayor diversidad que el ratio fijo de la BL.

Algorithm 4: Simulated Annealing

Entrada: problema, maxEvals, $\alpha = 0{,}999$
Salida: mejor solución encontrada

$T \leftarrow$ calibrarT_0(problema);
$s \leftarrow$ `crearSolución()`; $f \leftarrow$ `fitness(`s`)`;
$s^* \leftarrow s$; $f^* \leftarrow f$;
para $evals \leftarrow 1$ **to** $maxEvals - 1$ **hacer**
  $activas \leftarrow \{i : s[i] > 0\}$;
  $origen \leftarrow$ aleatorio en $activas$;
  $destino \leftarrow$ aleatorio en $\{1, \ldots, N\} \setminus \{origen\}$;
  $s' \leftarrow s$; $\delta \leftarrow$ uniforme(0,1, 0,9);
  $s'[origen] \mathrel{-}= \delta \cdot s[origen]$; $s'[destino] \mathrel{+}= \delta \cdot s[origen]$;
  `fix(`s'`)`; $f' \leftarrow$ `fitness(`s'`)`;
  si $f' > f$ ***o*** $aleatorio(0, 1) < e^{(f'-f)/T}$ **entonces**
    $s \leftarrow s'$; $f \leftarrow f'$;
    si $f > f^*$ **entonces**
      $s^* \leftarrow s$; $f^* \leftarrow f$;
  $T \leftarrow T \cdot \alpha$;
devolver s^*

4. Estructura del Código y Manual de Usuario

4.1. Estructura de directorios

```
portfolio/
  main.cpp                  # Punto de entrada, experimentos de 50 semillas
  CMakeLists.txt            # Sistema de construcción
  boxplot.py                # Script Python para gráficas de cajas
  data/
    ibex_35.csv             # 30 empresas IBEX 35, 2015-2025
    syp_100.csv             # 97 empresas S&P 100, 2015-2025
    syp_500.csv             # 457 empresas S&P 500, 2015-2025
  common/
```

```
    mh.h                    # Interfaz genérica MH<tDomain>
    problem.h               # Interfaz genérica Problem<tDomain>
    solution.h              # Typedefs tSolution, tFitness
    random.hpp              # Generador pseudoaleatorio de alta calidad
  inc/
    portfolioProblem.h      # Declaración del problema
    randomSearch.h          # Algoritmo aleatorio
    greedy.h                # Algoritmo greedy
    localsearch.h           # Búsqueda local
    simulatedAnnealing.h    # Simulated Annealing (voluntario)
  src/
    portfolioProblem.cpp    # Implementación: carga datos, matrices, fitness
```

4.2. Compilación

```
mkdir build && cd build
cmake ..
cmake --build .
```

4.3. Ejecución

El ejecutable `main` no requiere argumentos. Ejecuta automáticamente los cuatro algoritmos sobre los tres conjuntos de datos con 50 semillas cada uno, imprime las tablas de medias por pantalla y genera los ficheros CSV para boxplot:

```
cd build
cmake --build . && cd ..
./build/main
```

Al terminar se generan en el directorio raíz: `results_ibex_35.csv`, `results_syp_100.csv`, `results_syp_500.csv`.

Para generar los boxplots (requiere Python con `pandas` y `seaborn`):

```
python3 boxplot.py
```

Los parámetros configurables en `main.cpp` son `N_SEEDS` (número de semillas, por defecto 50) y `MAX_EVALS` (presupuesto de evaluaciones, por defecto 10 000). Las semillas utilizadas son $1, 2, \ldots, 50$, iguales para todos los algoritmos y conjuntos de datos.

5. Experimentos y Resultados

5.1. Casos del problema

Fichero	**Empresas** (N)	**Días train** (T)	**Días test**	**Rango** $[lo, hi]$
`ibex_35.csv`	30	$\sim$2560	254	[0,005, 0,08]
`syp_100.csv`	97	$\sim$2516	249	[0,005, 0,05]
`syp_500.csv`	457	$\sim$2516	249	[0,005, 0,02]

Parámetros comunes: $\lambda = 500$, `MAX_EVALS` $= 10\,000$, 50 semillas (1–50), $\alpha_{\text{SA}} = 0{,}999$.

5.2. Resultados

Las tablas muestran la media de las 50 ejecuciones. El fitness reportado para el Greedy es siempre el mismo al ser determinista.

Cuadro 1: Resultados IBEX 35 (medias, 50 semillas)

Algoritmo	**Fit. 2015–24**	**Fit. 2025**	**Ben. 2025**	**Evals**	**Tiempo (s)**
Greedy	−0,032	−0,019	−0,000	1	0,000
Random	−0,033	−0,022	0,000	10 000	0,282
BL	−0,030	−0,019	−0,000	10 000	0,259
SA	−0,030	−0,019	0,000	10 000	0,187

Cuadro 2: Resultados S&P 100 (medias, 50 semillas)

Algoritmo	**Fit. 2015–24**	**Fit. 2025**	**Ben. 2025**	**Evals**	**Tiempo (s)**
Greedy	−0,025	−0,031	0,000	1	0,001
Random	−0,026	−0,033	0,000	10 000	1,913
BL	−0,022	−0,027	0,000	10 000	4,634
SA	−0,022	−0,027	0,000	10 000	1,701

Cuadro 3: Resultados S&P 500 (medias, 50 semillas)

Algoritmo	**Fit. 2015–24**	**Fit. 2025**	**Ben. 2025**	**Evals**	**Tiempo (s)**
Greedy	−0,026	−0,031	0,000	1	0,038
Random	−0,028	−0,039	0,000	10 000	42,966
BL	−0,019	−0,028	0,000	10 000	199,447
SA	−0,019	−0,027	0,000	10 000	35,846

5.3. Boxplots

Las Figuras 1–3 muestran la distribución del fitness de entrenamiento en las 50 ejecuciones para cada conjunto de datos.

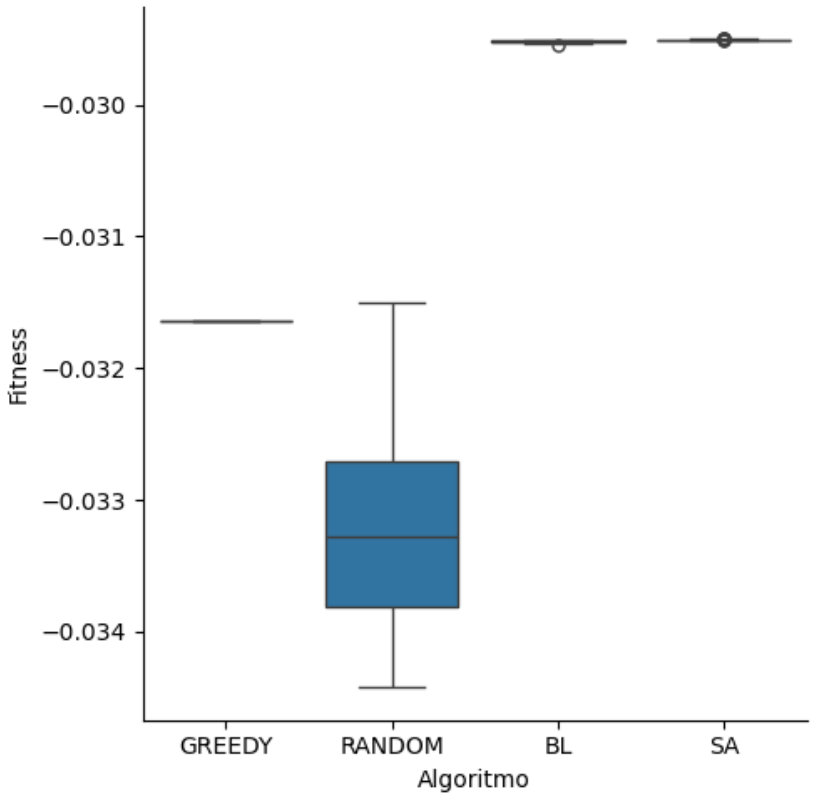


Figura 1: Distribución del fitness (2015–2024) en IBEX 35.

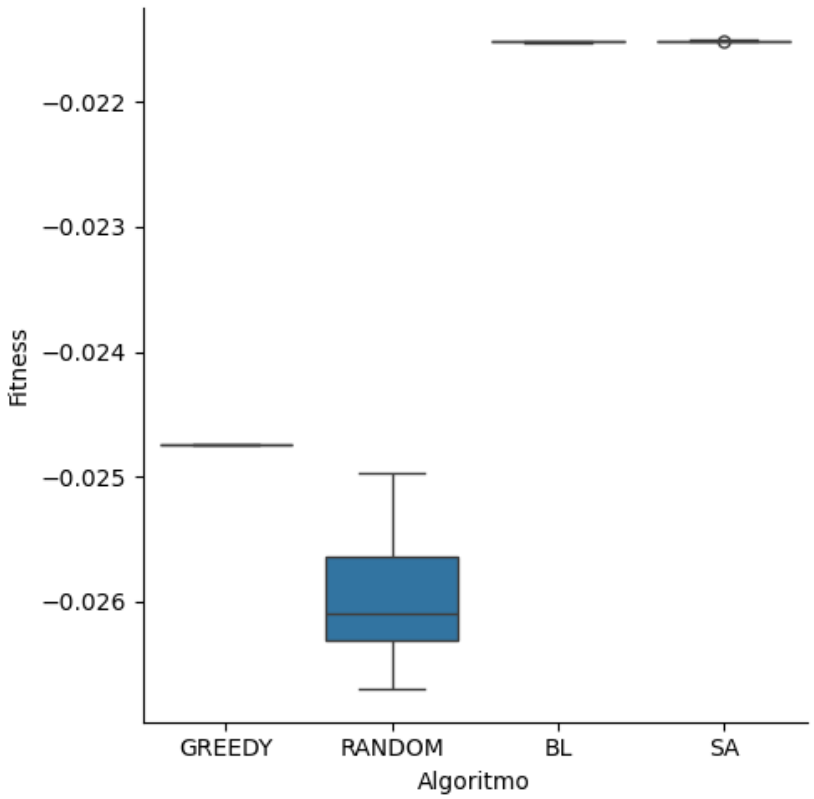


Figura 2: Distribución del fitness (2015–2024) en S&P 100.

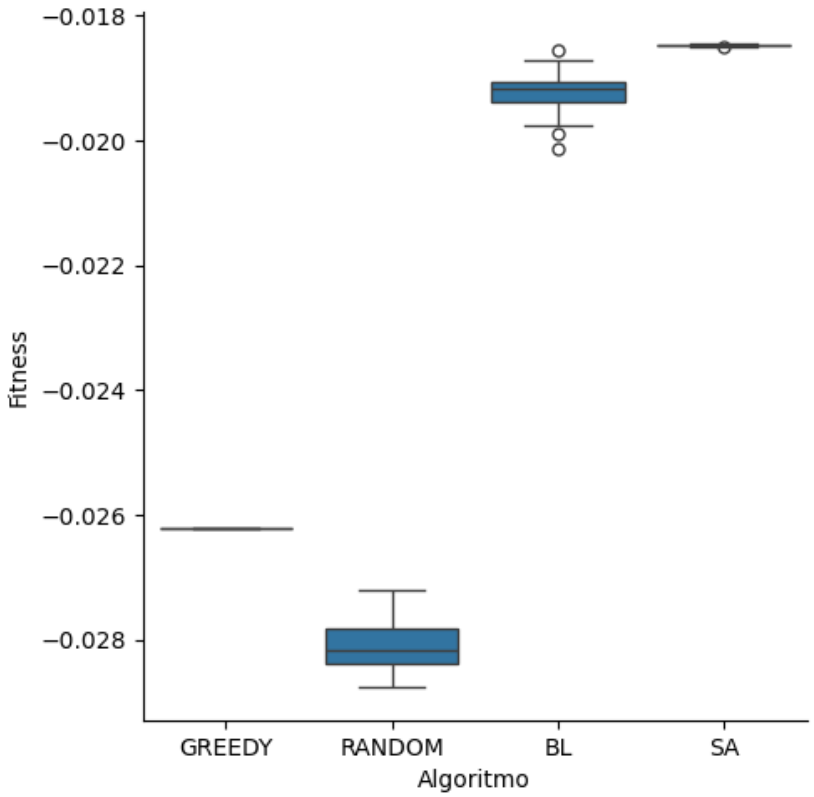


Figura 3: Distribución del fitness (2015–2024) en S&P 500.

6. Análisis de Resultados

6.1. Calidad de las soluciones

BL supera claramente a Random y Greedy. En ambos conjuntos con datos completos (IBEX 35 y S&P 100), la búsqueda local obtiene un fitness aproximadamente un 11 % mejor que el algoritmo aleatorio y un 7 % mejor que el greedy. Esto confirma que explotar la estructura local del espacio de búsqueda es efectivo para este problema.

El greedy ofrece una solución de calidad razonable con una única evaluación. Con solo 1 evaluación frente a las 10 000 del resto, la heurística basada en beneficio menos covarianza media supera al algoritmo aleatorio, lo que demuestra que la función heurística captura información relevante del problema.

SA y BL convergen a resultados prácticamente idénticos en IBEX 35 y S&P 100. Esto se explica porque con N pequeño (30 y 97 empresas) la BL ya es capaz de recorrer el vecindario ($N(N-1)$ vecinos) en pocas iteraciones antes de agotar el presupuesto, alcanzando óptimos locales de calidad comparable a los que encuentra el SA.

SA muestra mayor ventaja en S&P 500 ($N = 457$). Con un vecindario de $457 \times 456 \approx 208\,000$ pares, la BL apenas puede completar un recorrido completo en 10 000 evaluaciones y queda atrapada en el primer óptimo local encontrado. El SA, al aceptar soluciones peores con probabilidad controlada por la temperatura, puede escapar de esos óptimos y explorar regiones más prometedoras del espacio.

6.2. Variabilidad

La BL presenta muy baja varianza entre semillas en los tres conjuntos (los boxplots muestran cajas estrechas), lo que indica que el vecindario es suficiente para guiar la búsqueda hacia zonas similares independientemente del punto de partida. El SA hereda esta estabilidad y en S&P 500 converge consistentemente a valores de fitness similares entre semillas, algo que la BL no logra dado que su resultado depende en mayor medida del punto inicial.

El algoritmo aleatorio presenta la mayor varianza, especialmente en S&P 500 donde el espacio de búsqueda es enorme y 10 000 muestras aleatorias cubren una fracción ínfima de él.

6.3. Coste computacional y escalabilidad

Coste por evaluación. La operación dominante es el cálculo de `fitness`, que requiere $O(N^2)$ multiplicaciones (producto $w^\top \Sigma w$). El operador `fix` añade un coste $O(N)$ por llamada, pero se vuelve relevante cuando N es grande y el proceso de redistribución de pesos necesita más iteraciones para converger.

Escalado empírico con N. La Tabla **??** recoge los tiempos medios por semilla de cada algoritmo. Tomando S&P 100 como referencia y comparando con S&P 500 (N crece un factor $457/97 \approx 4{,}7$):

Cuadro 4: Tiempo medio por semilla (segundos) y factor de escala respecto a S&P 100

	IBEX 35 ($N = 30$)		**S&P 100** ($N = 97$)		**S&P 500** ($N = 457$)	
Alg.	t (s)	$\times 100$	t (s)	ref.	t (s)	$\times 100$
Greedy	0,000	—	0,001	1×	0,038	38×
Random	0,006	0,3×	0,038	1×	0,859	23×
BL	0,005	0,1×	0,093	1×	3,989	43×
SA	0,004	0,2×	0,034	1×	0,717	21×

El factor teórico esperado para un algoritmo cuyo coste por evaluación es estrictamente $O(N^2)$ sería $(457/97)^2 \approx 22$. Random y SA se aproximan a ese valor (23× y 21×), confirmando que su coste está dominado por `fitness`. La BL, en cambio, escala con un factor de 43×, sensiblemente mayor. La razón es que la BL llama a `fix` en *cada* vecino generado, y con $N = 457$ y `hi`=0,02 (máximo 50 activos simultáneos de 457), la redistribución de pesos en `fix` requiere muchas más iteraciones para alcanzar la factibilidad, elevando el coste efectivo por evaluación por encima del teórico $O(N^2)$.

El Greedy escala con un factor 38× de S&P 100 a S&P 500. Aunque el greedy solo hace una evaluación, necesita precalcular la heurística para cada empresa iterando sobre la fila correspondiente de Σ ($O(N^2)$ en total), lo que explica su tiempo no nulo en S&P 500.

Eficiencia calidad/tiempo. Una forma de comparar algoritmos con distinto presupuesto temporal es la mejora de fitness obtenida por unidad de tiempo. En S&P 500, la BL tarda 199,4 s por semilla y alcanza un fitness de $-0{,}019$, mientras que el SA obtiene el mismo fitness en solo 35,8 s, es decir, con **un coste 5,6 veces menor**. Esto se debe a la diferente estrategia de generación de vecinos: la BL construye y baraja los $N(N-1) \approx$

208 000 pares en cada iteración ($O(N^2)$ de overhead estructural), mientras que el SA selecciona aleatoriamente un par $(origen, destino)$ en $O(1)$, utilizando todo el presupuesto en evaluaciones de fitness en lugar de en gestión del vecindario.

Conclusión sobre eficiencia. Para instancias pequeñas ($N \lesssim 100$) los cuatro algoritmos son prácticamente inmediatos y BL y SA ofrecen idéntica calidad. Para instancias grandes ($N \sim 500$), el SA es la opción más eficiente: iguala la calidad de la BL consumiendo un quinto del tiempo, gracias a su generación de vecinos en $O(1)$ frente al $O(N^2)$ de estructurar el vecindario completo de la BL.

Referencias

[1] H. Markowitz, "Portfolio selection," *Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952.

[2] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[3] E. Konkov, *random.hpp: A C++11/14/17 header-only random number generator library*, GitHub, 2018.